

SRU	SR UNIVERSITY Campus Warangal	Program: II – B.Tech (CS & AI) Department: Computer Science and AI Semester: I
------------	---	---

AI ASSISTED CODING – ASSIGNMENT 12

Date: 21-09-2026

ASSIGNMENT 12 – AI ASSISTED CODING

Name	Gottiparthi Saikiran	Enrollment No.	2603A52L01
-------------	----------------------	-----------------------	------------

Submission Contents

1. AI-Assisted Bubble Sort Analysis
2. Linear Search vs Binary Search
3. Comparison of Sorting Algorithms
4. Optimising Student Search and Sort Program
5. Optimisation Challenge: Best Combination

QUESTION 1 – AI-Assisted Bubble Sort Analysis

Name: Gottiparthi Saikiran

Enrollment No.: 2603A52L01

University: SR University, Campus Warangal

1. Objective

Implement Bubble Sort in ascending order, count comparisons and swaps, explain one complete pass using AI, test multiple input lists, and compare the basic Bubble Sort with an early-exit optimized version.

2. Original Bubble Sort Implementation

```
numbers = [64, 34, 25, 12, 22, 11, 90]
comparison_count = 0
swap_count = 0

for i in range(len(numbers)):
    for j in range(len(numbers) - 1):
        comparison_count += 1
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
            swap_count += 1

print("Sorted list:", numbers)
print("Comparisons:", comparison_count)
print("Swaps:", swap_count)
```

3. Original Program Output

Sorted list: [11, 12, 22, 25, 34, 64, 90]

Comparisons: 42

Swaps: 14

4. AI Explanation – One Complete Pass

A complete pass means one run through the inner loop, comparing adjacent elements from left to right. For the first pass:

```
64 > 34 → swap → [34, 64, 25, 12, 22, 11, 90]
64 > 25 → swap → [34, 25, 64, 12, 22, 11, 90]
64 > 12 → swap → [34, 25, 12, 64, 22, 11, 90]
64 > 22 → swap → [34, 25, 12, 22, 64, 11, 90]
64 > 11 → swap → [34, 25, 12, 22, 11, 64, 90]
64 > 90 → no swap → [34, 25, 12, 22, 11, 64, 90]
```

After this pass, 90 is in its final position. The pass performs 6 comparisons and 5 swaps.

5. Testing Three Input Lists

Test	Input	Sorted List	Comparisons / Swaps
1	[64, 34, 25, 12, 22, 11, 90]	[11, 12, 22, 25, 34, 64, 90]	42 / 14
2	[1, 2, 3, 4, 5]	[1, 2, 3, 4, 5]	20 / 0
3	[5, 4, 3, 2, 1]	[1, 2, 3, 4, 5]	20 / 10

QUESTION 1 – AI-Assisted Bubble Sort Analysis (Continued)

6. AI Explanation – Early-Exit Optimization

The basic program always performs all passes even when the list becomes sorted. Early exit tracks whether any swap occurs during a pass. If a complete pass makes no swaps, the list is already sorted and the algorithm can stop.

For [1, 2, 3, 4, 5], the original version performs 20 comparisons. With early exit, the first pass performs only 4 comparisons and makes no swaps, so the algorithm stops.

7. Original vs Early-Exit Version

Input	Version	Comparisons	Swaps	Sorted Result
Original list	Original	42	14	[11, 12, 22, 25, 34, 64, 90]
Original list	Early Exit	36	14	[11, 12, 22, 25, 34, 64, 90]
Sorted list	Original	20	0	[1, 2, 3, 4, 5]
Sorted list	Early Exit	4	0	[1, 2, 3, 4, 5]
Reverse list	Original	20	10	[1, 2, 3, 4, 5]
Reverse list	Early Exit	20	10	[1, 2, 3, 4, 5]

8. Observation / Conclusion

The Bubble Sort implementation correctly sorted all three input lists and counted comparisons and swaps. The early-exit optimization reduced comparisons for the original and already-sorted lists. For the reverse-sorted list, comparisons remained the same. The optimization improves best-case work from quadratic to linear, while worst-case work remains quadratic.

9. Verification

Syntax check passed and both the original and early-exit versions were executed. The sorted results were the same, verifying that the optimization preserved the sorting behavior.

QUESTION 2 – Linear Search vs Binary Search

Name: Gottiparthi Saikiran

Enrollment No.: 2603A52L01

University: SR University, Campus Warangal

1. Objective

Implement Linear Search and Binary Search, compare their number of comparisons, test different target positions, explain why Binary Search requires sorted data, compare their use cases, and implement a valid optimization.

2. Initial Program

The original starter program contained the sorted list, target value, and TODO comments for implementing Linear Search and Binary Search. It was run before modifications and produced no output, as expected.

3. Basic Search Implementation and Initial Result

Copilot implemented straightforward Linear Search and Binary Search. Both reported the zero-based position and comparison count. For target 45, both found the target at index 5. Linear Search used 6 comparisons and Binary Search used 5 comparisons.

4. Four Required Test Cases

Target	Linear Search	Binary Search	Purpose
5	Position 0; 1 comparison	Position 0; 5 comparisons	First element
31	Position 4; 5 comparisons	Position 4; 1 comparison	Middle element
89	Position 8; 9 comparisons	Position 8; 7 comparisons	Last element
100	Not found; 9 comparisons	Not found; 8 comparisons	Absent element

5. Why Binary Search Requires Sorted Data

Binary Search uses the midpoint value to decide which half of the list can be discarded. For the sorted list [5, 12, 18, 23, 31, 45, 56, 72, 89], when searching for 45, the middle value is 31. Since 45 is greater than 31, the values before 31 can safely be ignored.

If the list is unsorted, this decision is not reliable. For example, in [5, 12, 18, 23, 72, 45, 56, 31, 89], the midpoint is 72. Since 45 is less than 72, Binary Search would search the left side even though 45 is actually on the right side. Therefore, Binary Search requires sorted data.

6. Linear Search vs Binary Search – Use Cases

For a single search on unsorted data, sorting first may cost more than scanning. For repeated searches on the same data, sorting once and using Binary Search can be beneficial.

QUESTION 2 – Linear Search vs Binary Search (Continued)

7. AI-Suggested Optimization

AI suggested allowing Linear Search to stop early when the current value is greater than the target. Because the list is sorted, the target cannot occur later once a larger value has been reached.

The optimization depends on sorted data. It remains $O(n)$ in the worst case and uses $O(1)$ extra space.

8. Optimized Linear Search – Fair Comparison

Target	Original Linear Search	Optimized Linear Search	Observation
5	Position 0; 1 comparison	Position 0; 1 comparison	No improvement needed.
31	Position 4; 5 comparisons	Position 4; 5 comparisons	Target is found before early stop can help.
89	Position 8; 9 comparisons	Position 8; 9 comparisons	Target is near the end.
100	Not found; 9 comparisons	Not found; 9 comparisons	Target is beyond all values.
20	Not found; 9 comparisons	Not found; 4 comparisons	Stops at 23 because $23 > 20$.

9. Linear Search vs Binary Search

Feature	Linear Search	Binary Search
How it works	Checks elements one by one from the start.	Checks the middle and repeatedly discards half the remaining range.
Data requirement	Works with sorted or unsorted data.	Requires sorted data.
Time complexity	$O(n)$	$O(\log n)$
Best suited for	Small/unsorted datasets or occasional searches.	Large, sorted datasets, especially repeated searches.

10. Observation / Conclusion

Both search algorithms correctly found the required targets or reported them as absent. Linear Search is flexible because it does not require sorted data, while Binary Search uses sorted order to reduce the search range and has $O(\log n)$ search time. For target 20, the early-stop optimization reduced examined elements from 9 to 4.

11. Verification

The search program was executed after each modification. The results were checked for the first, middle, last, and absent targets, and the optimization was tested with target 20.

QUESTION 3 – Comparison of Sorting Algorithms

Name: Gottiparthi Saikiran

Enrollment No.: 2603A52L01

University: SR University, Campus Warangal

1. Objective

Implement Bubble Sort, Selection Sort, and Insertion Sort using the same input, compare their operation counts, study their strengths and weaknesses, test nearly sorted data, and compare experimental results with theoretical time complexity.

2. Input Data

[45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

3. Implementation

Three straightforward sorting algorithms were implemented. Each algorithm sorts a fresh copy of the same input so that the results are directly comparable. Bubble Sort and Selection Sort count comparisons and swaps. Insertion Sort counts comparisons and shifts.

4. Original Input – Experimental Results

Algorithm	Sorted Result	Comparisons	Swaps / Shifts
Bubble Sort	[5, 10, 12, 23, 34, 45, 56, 67, 78, 89]	45	24 swaps
Selection Sort	[5, 10, 12, 23, 34, 45, 56, 67, 78, 89]	45	6 swaps
Insertion Sort	[5, 10, 12, 23, 34, 45, 56, 67, 78, 89]	31	24 shifts

5. AI Analysis – Strengths and Weaknesses

Algorithm	Strengths	Weaknesses	Theoretical Complexity
Bubble Sort	Simple; stable; swaps neighboring out-of-order values.	Many comparisons and potentially many swaps.	$\Theta(n^2)$ best, average, worst as implemented.
Selection Sort	Simple; few swaps; useful when writes are costly.	Scans the remaining unsorted portion; generally not stable.	$\Theta(n^2)$ best, average, worst.
Insertion Sort	Effective for small or nearly sorted lists; stable.	Can require many shifts when values are far from final positions.	$\Theta(n)$ best; $\Theta(n^2)$ average, worst.

QUESTION 3 – Comparison of Sorting Algorithms (Continued)

6. Nearly Sorted Data Test

Nearly sorted input: [5, 10, 12, 23, 34, 45, 56, 78, 67, 89]

All three algorithms produced: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Algorithm	Comparisons	Swaps / Shifts
Bubble Sort	45	1 swap
Selection Sort	45	1 swap
Insertion Sort	10	1 shift

7. Experimental Results vs Theoretical Complexity

The operation counts are specific to these two 10-element inputs and are not elapsed-time measurements.

Bubble Sort made 45 comparisons for both inputs because the implementation has no early-exit optimization. Its swaps decreased from 24 to 1.

Selection Sort also made 45 comparisons for both inputs because every pass scans the remaining unsorted portion. Its swaps decreased from 6 to 1.

Insertion Sort decreased from 31 comparisons and 24 shifts to 10 comparisons and 1 shift on the nearly sorted input. This supports its $\Theta(n)$ best-case behavior and explains why it is effective for nearly sorted data.

8. Observation / Conclusion

All three algorithms correctly sorted both input lists. Bubble Sort and Selection Sort continued to perform the same number of comparisons on the nearly sorted input, while Insertion Sort benefited substantially from the nearly sorted data. Therefore, Insertion Sort is particularly suitable for small or nearly sorted datasets.

9. Verification

Syntax validation passed and the program was executed successfully. Each algorithm used a separate copy of the input, and all three produced the same sorted results for both tests.

QUESTION 4 – Optimising Student Search and Sort Program

Name: Gottiparthi Saikiran

Enrollment No.: 2603A52L01

University: SR University, Campus Warangal

1. Objective

Find the highest-mark student, sort students by mark, search by name, identify unnecessary work using AI, and implement an improved approach while comparing it with the straightforward solution.

2. Student Data

```
students = [("Arun", 82), ("Bala", 65), ("Chitra", 91),  
            ("Divya", 74), ("Esha", 88)]
```

3. Straightforward Solution

Three functions were implemented: finding the highest-mark student, sorting students by mark in ascending order, and searching for a student by name. The search was tested with Chitra.

Results: Highest-mark student = Chitra (91). Search result for Chitra = ('Chitra', 91). The students were correctly sorted in ascending order.

4. AI Analysis of Unnecessary Work

The individual functions were already simple. However, when both the maximum and sorted results are required, finding the maximum and then sorting performs separate work. The highest-mark student can be obtained from the final element of the sorted result.

For repeated name searches, scanning the list each time repeats $O(n)$ work. A dictionary can be built once for average $O(1)$ name lookups. For a single search, the straightforward scan is simpler and avoids extra storage.

5. AI-Suggested Improved Approach

The improved approach builds a name lookup dictionary, sorts the students once, and obtains the highest-mark student from the final element of the sorted list.

Building the dictionary takes $O(n)$ time and sorting takes $O(n \log n)$, so the overall time remains $O(n \log n)$. The dictionary and sorted copy use $O(n)$ additional space. Name lookups are $O(1)$ on average after the dictionary is built.

6. Original vs Improved Results

Requirement	Original	Improved
Highest mark	Chitra (91)	Chitra (91)
Sort by mark	Correct ascending order	Same correct ascending order
Search Chitra	Chitra (91)	Chitra (91)
Repeated name searches	$O(n)$ per search	$O(1)$ average after dictionary build
Overall sorting-based time	$O(n \log n)$	$O(n \log n)$

7. Observation / Conclusion

The improved solution preserved the behavior of the straightforward solution while reducing repeated work when many name searches are required. Sorting remains the dominant $O(n \log n)$ operation. The dictionary is most useful for repeated searches; for one search, the original linear scan is simpler and uses less extra space.

8. Verification

Both original and improved versions were executed successfully. They produced the same highest-mark student, sorted order, and search result for Chitra. Syntax and runtime checks passed.

QUESTION 5 – Optimisation Challenge: Best Combination

Name: Gottiparthi Saikiran

Enrollment No.: 2603A52L01

University: SR University, Campus Warangal

1. Objective

Find a combination of items whose total price is as high as possible without exceeding the budget. Compare Brute Force, Greedy, and Dynamic Programming, determine which approaches guarantee an optimal result, implement a guaranteed optimal approach, test multiple budgets, and analyse complexity and possible optimizations.

2. Input Data

[("Laptop", 60000), ("Phone", 30000), ("Tablet", 20000), ("Watch", 10000), ("Headphones", 5000)]

Initial budget: ₹50,000

3. Straightforward Brute-Force Approach

The brute-force solution checks every possible combination of items, keeps the combination with the highest total that does not exceed the budget, and displays the selected items, total price, and remaining budget.

For ₹50,000, the selected combination is Phone + Tablet. Total = ₹50,000; Remaining budget = ₹0.

Because there are n items, the approach considers 2^n subsets. Summing the items in a subset can take $O(n)$, giving $O(n \cdot 2^n)$ time and $O(n)$ auxiliary space for storing the current/best combination.

4. AI Comparison – Brute Force vs Greedy vs Dynamic Programming

Approach	How it works	Time	Space	Optimal?
Brute Force	Checks every subset and keeps the best feasible combination.	$O(n \cdot 2^n)$	$O(n)$	Yes
Greedy	Makes locally best choices, such as selecting a high-value item first.	$O(n \log n)$	$O(n)$ or $O(1)$ extra	No, not in general
Dynamic Programming	Builds optimal results for smaller budget capacities and items.	$O(nB)$	$O(nB)$ in current implementation	Yes

5. Why Greedy Does Not Guarantee the Optimum

For example, with budget 10 and item prices 6, 5, and 5, a greedy choice of 6 leaves 4 and produces total 6. The optimal choice is $5 + 5 = 10$. Therefore, Greedy is not guaranteed to solve the general 0/1 combination problem optimally.

6. Selected Optimal Approach

Dynamic Programming was implemented as the scalable optimal approach for the given problem. Brute Force was retained as a reference so that its optimal result could be compared with the DP result.

QUESTION 5 – Optimisation Challenge: Best Combination (Continued)

7. Test Results

Budget	Optimal Total	Selected Combination
■50,000	■50,000	Phone + Tablet
■65,000	■65,000	Laptop cannot fit; optimal combination reaches ■65,000
■10,000	■10,000	Watch + Headphones

The brute-force and Dynamic Programming results matched for all three budgets. Assertions also verified that the budget was not exceeded and no item was selected more than once.

8. Complexity and Possible Optimizations

Brute Force: $O(n \cdot 2^n)$ time and $O(n)$ auxiliary space. Dynamic Programming: $O(nB)$ time and $O(nB)$ space in the current table-based implementation, where B is the budget.

For larger inputs, meet-in-the-middle can reduce the dependence on n for some cases. Here, all prices are multiples of ■5,000, so prices and budgets can be scaled down by ■5,000 before running DP. A one-dimensional DP table can reduce value-table memory to $O(B)$; additional choice tracking is needed if the selected items must be reconstructed.

9. Observation / Conclusion

The brute-force and Dynamic Programming approaches both produced the same optimal totals for all three tested budgets. Both guarantee optimality for this 0/1 combination problem, whereas a Greedy strategy does not guarantee an optimal result in general. Brute Force is simple but exponential, while Dynamic Programming is more scalable when the numeric budget is moderate.

10. Verification

Syntax validation and runtime checks passed. The algorithms were tested at ■50,000, ■65,000, and ■10,000, and their optimal totals matched. Assertions verified the budget limit and absence of duplicate item selections.